Running head: Agentic ODE Parameter Estimation

An Agentic AI Workflow for Parameter Estimation of Complex Differential Equation Systems

Saakaar Bhatnagar[1]

[1] *<Affiliation, City, State/Country>*
ORCID: <ORCID ID>

Corresponding author: Saakaar Bhatnagar, <email address>

Abstract

Parameter estimation for ordinary differential equation (ODE) models fitted to experimental data is an iterative process often made difficult by heuristic fitting methods that make severely limiting assumptions, noisy experimental data, large search spaces and stiff ODE dynamics. Posing the estimation problem as an optimization problem is a commonly used method, but usually needs advanced knowledge of optimization algorithms and is prone to implementation errors and inefficiencies. This chapter describes an AI agent centered workflow for converting a lightweight estimation problem specification into a reproducible calibration pipeline. The overall task of estimation is broken down into sub steps, with the human in the loop confirming the agent's work. The agent's context for each step is populated with sub task-specific knowledge, enabling expert execution instead of random guesswork. The user provides key definitions in a structured file system. The workflow emphasizes reproducibility and audit through inspection of generated code, and diagnostic checks that help distinguish numerical, modeling, and data-quality failures. Overall, this workflow allows easy and robust iteration on model form, search space and objective functions, which critically affects the obtained models.

Key words

Parameter Estimation; Ordinary Differential Equations; Global Optimization; Differentiable Programming; Gradient-Based Optimization

1 Introduction

1.1 Motivation

add shaowu and nithin cites

Ordinary differential equation (ODE) models are widely used to represent biological, chemical, and physical systems, these often are put into bigger models, eg battery

A common strategy is to frame the fitting problem as an optimization problem, by minimizing the solution of the ODE or some derivative of it to the experimental data. include citations. Usually, practitioners employ global search methods, where the search is driven by population methods such as swarm optimization, differential evolution or genetic algorithms. These methods are agnostic to the physics of the problem, are trivially parallelizable and are robust to failure of samples.

These methods return a good initial guess to the problem, which can be fine-tuned using gradient based methods to obtain a more accurate fit. An additional challenge for gradient based methods is that for numerical derivatives of the solution w.r.t parameters may be highly unstable or inaccurate. This is dependent on the numerical properties of the ODE, for eg. stiffness. Recent work has shown that derivatives obtained via automatic differentiation of the solver are more stable and can be used to get accurate derivatives of solutions w.r.t parameters, which can be used for parameter updates, similar to the methods of adjoints refine and cite

[1–3]

Hence, a typical calibration exercise requires the user to translate equations into code, connect the code to observation data, define parameter bounds and transformations, select suitable numerical solvers, choose estimation algorithms, and diagnose failures when the fit is poor.

Executing all of the above steps requires expertise and effort to setup the entire estimation process. Any of these steps, if done incorrectly, can cause failure of the estimation problem. Some common failure modes include include (inappropriate scaling, incorrect optimization setup, insufficient estimation algorithms, poor objective function construction, insufficient constraint flexibility, too broad or too narrow search ranges , inefficient code). However, with expert oversight and assistance, this methodology can be reliable at estimating the required parameters.

1.2 Agentic AI workflows

AI agents have recently been adopted across scientific computing for tasks such as geometry and CAD creation [4, 5], solver orchestration [6, 7], end-to-end simulation pipelines and post-processing [8, 9]. These workflows share a characteristic profile: they require substantial manual setup, proceed largely by trial and error, expose a large number of configuration knobs that materially affect the solution yet are easily overlooked, and demand expert knowledge to operate the underlying tools effectively. Crucially, their outcomes are often *verifiable*- the task's output can be checked against clear evidence, rules, tests, or ground truth rather than judged only by subjective preference. This combination of high manual overhead, expertise requirements, and built-in verifiability makes such workflows particularly amenable to augmentation by AI agents: the agent can shoulder the routine and knowledge-intensive setup. The success of these efforts, together with the recurring need for manual intervention and domain expertise, directly motivates applying AI agents to the ODE parameter estimation problem considered here.

However, delegating complex problems to an AI agent without user oversight risks hallucination, context degradation, and silent, incorrect assumptions, any of which can yield erroneous solutions [8, 10]. An agent left to run unsupervised through a long chain of decisions can therefore accumulate and amplify errors that a domain expert would catch immediately. Add context length limit. A potential solution to these problems is using models with large parameter counts and context windows, recursive thinking and more. Models provided by large model providers using a cloud model can be used to solve such problems via effective tool calling and the ability to directly process a large number of tokens. However, this methodology comes with issues around

reproducibility and auditability, where an hosted model may make silent assumptions on different runs of the same problem. More importantly, using cloud providers to solve problems in scientific computing comes with privacy and intellectual property concerns, and often, practitioners insist on their data, mathematical models and workflow knowledge not leaving air-gapped computing environments that are locally managed.

The workflow described in this chapter is designed around these constraints, and describes a method to orchestrate `local` coding models to solve ODE parameter estimation problems. The broader fitting problem is decomposed into a sequence of sub-steps, each completed by the agent and producing intermediate artifacts written to disk. The decomposition enables effective utilization of context windows, and the tool performs a series of deterministic and LLM-based evaluations at every step. The tool identifies inconsistencies in the problem specification, repairs common input pathologies, transforms user-defined model functions into JIT-compilable (just-in-time compiled) routines, and assembles a two-stage optimization pipeline that combines a parallelized global search with gradient-based refinement.

This chapter links two repositories: NAME1 which orchestrates the workflow as a set of skill files (utilizing remote LLMs), and NAME2 which runs an LLM locally. The workflow described in the chapter is focused on NAME2 since it has more limitations

2 Methods

2.1 Context Assisted Agents via Skills

A transformer language model defines an autoregressive distribution $p(x_t \mid x_{<t})$ over the next token x_t given all preceding tokens in its context window, repeatedly predicting the next piece of text from everything written so far. This conditioning is implemented by a mechanism called self-attention: as the model processes each token, it scores how relevant every earlier token is to the one currently being processed, using this relevance score to predict the next token. The model has no persistent task-specific memory beyond this: everything that shapes a prediction must be present, as tokens, in the context. All task-relevant behavior is therefore induced at inference time by the context: the instructions, reference material, and examples assembled into the input are the primary levers for controlling an agent.

The context window is also a finite, shared resource, and attention makes its use non-uniform. For each token, the relevance scores described above are normalized so that they sum to one across all the earlier tokens (an operation called a softmax). Adding more tokens to the context window therefore forces the few that actually matter to share this fixed budget with a larger pool of competitors. Loading an entire domain into one static prompt thus scales poorly: it inflates cost and latency, pushes against the length limit, and tends to lower quality as the signal-to-noise ratio of the window falls.

Hence, each sub-step of the process is scoped to one sub-task, that determines which material is loaded into the context window precisely when that step executes: the rules the step must follow, the structural templates its output must match, and tool calls the LLM may make. Because the model attends over the sub-task, detail belonging to other steps is kept out of the window and out of the attention budget. This also keeps the active window dense with relevant tokens and well within its length limit. This both sharpens output quality and lets each step be revised independently by

editing its context alone.

2.2 Search Algorithms

2.2.1 Population methods

The framework ships with two population-based global optimizers, Particle Swarm Optimization (PSO) and Differential Evolution (DE), for the coarse search that precedes gradient-based refinement. Both are *gradient-free*, tolerating the stiff, non-smooth losses typical of ODE fitting; *global*, exploring an entire population rather than a single iterate to avoid poor local minima; and *embarrassingly parallel*, since per-candidate evaluations are independent and dispatched in parallel. The population is given a space-filling initialization via an optimized Latin Hypercube (LHS) design [11], and both search in the normalized $[-1,1]^d$ space so parameters spanning many orders of magnitude are treated on equal footing. The best candidate found seeds the gradient stage.

Particle Swarm Optimization (PSO) The global search defaults to PSO[12], a population-based metaheuristic in which a swarm of N candidate solutions (*particles*) explores the normalized parameter space $[-1,1]^d$. Particle i carries a position $\mathbf{x}_i \in [-1,1]^d$ and a velocity $\mathbf{v}_i$. The swarm is seeded with a LHS design to spread the initial positions evenly, and a star topology is used so every particle shares a single global best.

Let $f(\cdot)$ denote the (normalized) loss. Each particle tracks its own best position $\mathbf{p}_i$ and the swarm's global best $\mathbf{g}$:

$$\mathbf{p}_i \leftarrow \arg\min_{\mathbf{x}\in\{\mathbf{p}_i,\mathbf{x}_i\}} f(\mathbf{x}), \qquad \mathbf{g} \leftarrow \arg\min_i f(\mathbf{p}_i). \tag{1}$$

At each iteration k the velocity and position are updated by

$$\mathbf{v}_i^{k+1} = w\,\mathbf{v}_i^k + c_1\mathbf{r}_1 \odot (\mathbf{p}_i - \mathbf{x}_i^k) + c_2\mathbf{r}_2 \odot (\mathbf{g} - \mathbf{x}_i^k), \tag{2}$$

$$\mathbf{x}_i^{k+1} = \mathbf{x}_i^k + \mathbf{v}_i^{k+1}, \tag{3}$$

where $\mathbf{r}_1, \mathbf{r}_2 \sim \mathscr{U}(0,1)^d$ are drawn element-wise, $\odot$ is the Hadamard product, w is the inertia weight, and c_1, c_2 are the cognitive and social coefficients. The velocity is clamped component-wise to $[-0.3, 0.3]$, and particles leaving the box are projected back to the nearest boundary.

Relative to differential evolution, PSO tends to converge faster on easier problem landscapes cite: the velocity memory and shared global attractor let information propagate through the swarm quickly, so good regions are exploited in fewer loss evaluations.

Differential Evolution (DE) As an alternative global search, the framework provides Differential Evolution [13], a population-based evolutionary algorithm well suited to multimodal, non-differentiable objectives. A population of N candidate vectors $\{\mathbf{x}_i\}_{i=1}^N$ in $[-1,1]^d$ is initialized with LHS and evolved by mutation, crossover, and selection.

For each target vector $\mathbf{x}_i$ at generation k, a *mutant* is formed from the current best and two distinct randomly chosen members ($r_1 \neq r_2 \neq i$):

$$\mathbf{m}_i = \mathbf{x}_{\text{best}}^k + F\left(\mathbf{x}_{r_1}^k - \mathbf{x}_{r_2}^k\right), \tag{4}$$

where the differential weight F is sampled per generation from $\mathscr{U}(0.5, 1.0)$ to improve robustness. A trial vector $\mathbf{u}_i$ is built by binomial crossover, mixing the mutant with the target at a per-component rate set by the recombination constant $CR = 0.7$:

$$u_{i,j} = \begin{cases} m_{i,j}, & \text{if } \rho_j \le CR \text{ or } j = j_{\text{rand}}, \\ x^k_{i,j}, & \text{otherwise}, \end{cases} \qquad \rho_j \sim \mathscr{U}(0,1), \tag{5}$$

where j_{rand} guarantees at least one component is taken from the mutant. Greedy selection retains whichever of the trial or target has lower loss:

$$\mathbf{x}_i^{k+1} = \begin{cases} \mathbf{u}_i, & \text{if } f(\mathbf{u}_i) \le f(\mathbf{x}_i^k), \\ \mathbf{x}_i^k, & \text{otherwise.} \end{cases} \tag{6}$$

Relative to PSO, DE is more robust on highly multimodal or ill-conditioned landscapes[13]: its difference-vector mutation adapts the search scale to the population's current spread, and the greedy, elitist selection prevents the population from drifting away from good solutions. It is therefore the safer choice when PSO stalls in a poor local minimum, at the cost of more loss evaluations to converge.

2.2.2 Gradient-based methods

add references on building differentiable models and their recent use in optimization and RL pipelines

The best candidate from the global search is refined by a gradient-based optimizer that minimizes the same normalized loss $f(\mathbf{x})$ over $[-1,1]^d$. Gradients are obtained by *automatic differentiation* (AD) through the full discretized ODE solve, rather than by finite differences. Because AD differentiates the discretized solver–objective map itself, it yields exact discrete sensitivities and requires no perturbation step size—a quantity that is notoriously hard to choose for stiff or highly nonlinear systems [14, 15].include a parallel to discrete adjoints The update is a projected step that keeps iterates inside the search box,

$$\mathbf{x}^{k+1} = \Pi_{[-1,1]^d}\big(\mathbf{x}^k + \Delta\mathbf{x}^k\big), \qquad \Delta\mathbf{x}^k = \mathscr{U}\left(\nabla f(\mathbf{x}^k), \mathbf{x}^k\right), \tag{7}$$

where Π is clipping onto the bounds and $\mathscr{U}$ is the optimizer's update rule. The framework exposes both a first-order method (Adam, with an exponentially decaying learning-rate schedule) and a second-order quasi-Newton method (L-BFGS); the best iterate is tracked throughout and returned, and a failed integration is caught so the search exits gracefully with the best solution so far.

This stage is intended for *fine-tuning* rather than global discovery: it converges quickly but needs a good initial guess, which the population-based search provides. It is resource-efficient, optimizing a single iterate instead of a whole population, and scales well with dimension, since AD computes the full gradient at a cost comparable to a single loss evaluation regardless of the number of parameters—unlike finite differences, whose cost grows linearly with dimension.

The gradient based search is also the more fragile of the two methods, and several failure modes are worth noting. The computed gradient inherits the accuracy of the forward solve, so loosely converged or under-resolved integration injects noise into ∇f and can cause the iterates to diverge; the solver tolerances must generally be tighter for the gradient stage than for the global

search to allow for a stable adjoint computation. Excessively noisy data has a similar effect, producing a rugged loss surface whose local gradients point away from the true optimum. Some systems are ill-suited to differentiation: non-smooth dynamics (event triggers, clipping, piecewise or discontinuous right-hand sides) break differentiability. Even for smooth, well-behaved systems, convergence can stall on *sloppy* or ill-conditioned problems where the loss has long flat valleys (near-degenerate parameter combinations), and a poor initial guess can leave the optimizer in the wrong basin since the local search cannot escape it. Finally, stiff systems may drive the adaptive solver to exceed its step budget, causing an integration failure mid-search; the framework guards against this by tracking the best iterate and exiting gracefully when a solve fails.

2.3 Post-Fit Sloppiness Diagnostic

After the gradient stage converges, the framework automatically characterizes how well the data constrain the fit by computing the local curvature of the loss at the optimum—the *sloppiness* spectrum in the sense of Gutenkunst *et al.* and Hass *et al.* [16, 17]. The analysis is performed in the coordinate the search actually uses: let $\mathbf{u}$ denote the per-parameter axis with $u_i = \log_{10}\theta_i$ for parameters flagged `LOGSCALE` and $u_i = \theta_i$ otherwise, so curvature in a log-scaled parameter is measured per decade. Writing the converged best fit as $\mathbf{u}^\star$, the framework forms the Hessian of the normalized loss,

$$H \;=\; \nabla^2_{\mathbf{u}} f(\mathbf{u}^\star) \;\in\; \mathbb{R}^{d\times d}, \tag{8}$$

by second-order automatic differentiation through the entire ODE solve, falling back to a symmetric finite difference of the (first-order AD) gradient when higher-order differentiation through the stiff implicit solver is unavailable. The Hessian is symmetrized, $H \leftarrow \frac{1}{2}(H + H^\top)$, and diagonalized,

$$H \;=\; V\Lambda V^\top, \qquad \Lambda = \mathrm{diag}(\lambda_1 \geq \cdots \geq \lambda_d), \tag{9}$$

yielding an eigenvalue spectrum $\{\lambda_k\}$ and orthonormal eigenvectors $\{\mathbf{v}_k\}$. Each eigenvector is a *direction* in parameter space (a linear combination of the u_i) and its eigenvalue is the curvature of the loss along that direction: large λ_k marks a *stiff*, well-constrained combination, small λ_k a *sloppy*, poorly-constrained one, and $\lambda_k \to 0$ a flat, practically non-identifiable direction.

The reported verdict uses the eigenvalues normalized by the largest, $\hat{\lambda}_k = \lambda_k/\lambda_1$. A direction is flagged non-identifiable when $\hat{\lambda}_k$ falls below a floor $\varepsilon_0 = 10^{-6}$, and the fit is termed *sloppy* when the identifiable modes span more than six orders of magnitude,

$$\log_{10}\!\left(\frac{\lambda_1}{\lambda_{\min}^{\text{ident}}}\right) > 6 \qquad \text{or} \qquad \exists k : \hat{\lambda}_k < \varepsilon_0, \tag{10}$$

where $\lambda_{\min}^{\text{ident}}$ is the smallest eigenvalue above the floor.

This is a local, quadratic measure valid only at a converged optimum, so it detects the flat valleys that stall gradient descent but does not, by itself, map the global extent of a non-identifiable valley, i.e, it is a measure of the quality of the current fit, and not whether there may be a better global minimum at some other solution.

3 Procedure

This section describes how to configure and run an ODE parameter-estimation problem using the local pfit workflow. The supported starting point is a reviewed mathematical problem description in `inputs/user_info.txt`, together with one or more CSV data files. The user is responsible for specifying the equations, parameter bounds, initial conditions, data mappings, and mathematical objective. The framework then uses a locally hosted Ollama coding model to help convert this specification into executable model code.

The workflow is divided into five command-line stages:

`pfit new` → `pfit check` → `pfit jax` → `pfit run` → `pfit diagnose`

During `pfit new`, the configured local coding model extracts a structured fitting problem from `user_info.txt` and the declared CSV files. Deterministic renderers and validators then control whether `inputs/user_input.yaml` and `generated/user_model.py` are accepted. The `pfit check` stage examines the resulting data, configuration, model declarations, and objective, and writes a report without modifying the scientific model.

The `pfit jax` stage uses the local coding model to translate the accepted Python model into JAX. The candidate translation is subjected to generated-code interface checks, numerical smoke tests, sampled source-to-JAX comparisons, and source-freshness checks. The `pfit run` stage repeats deterministic readiness checks and performs population-based exploration followed by gradient-based refinement. Each invocation creates a new preserved run directory, and post-fit sloppiness analysis is performed by default.

Finally, `pfit diagnose` reconstructs a completed run from its saved snapshot, calculates residual and optimization diagnostics, and generates measured-versus-simulated and residual plots. It also summarizes the sloppiness results produced during fitting. The configured local model may be used to interpret the computed evidence, but it does not modify the session.

The local language model does not independently determine whether generated scientific code is correct. It proposes structured model content and JAX translations within bounded stages, while deterministic code owns validation, file writing, source-consistency checks, and numerical acceptance. These checks reduce common implementation errors, but they do not prove that the user's scientific model is appropriate or that a generated translation is mathematically equivalent for every possible state and parameter value.

The separation between stages is deliberate. Each stage produces files that can be inspected before the next stage begins, making assumptions and generated code visible to the user. A successful command means that the artifact passed the checks implemented for that stage; it does not establish optimizer convergence, global optimality, parameter identifiability, or scientific adequacy of the model. These questions require examination of the fitted trajectories, residuals, optimizer history, parameter bounds, gradient evidence, and sloppiness results produced later in the workflow.

The following subsections first define the software, data, and problem- specification requirements and then describe each command and its generated artifacts in detail.

Before starting

Need to describe local model setup, make sure the env setup is correct, The framework should be installed in a Python 3.12 virtual environment. The package and its direct dependencies are defined in `pyproject.toml`. For a reproducible environment using the versions tested with the repository, install the lock file:

```
python -m pip install -r requirements-lock.txt
```

The local coding model is served through Ollama. However, once the model setup is complete, the local coding model is no longer required. Hence, if a user wishes to iterate using different settings, they do not need to rerun the coding model, or need GPU access.

A new problem begins with the following directory structure:

```
sessions/<session-name>/
  inputs/
    user_info.txt
    <data-file>.csv
```

The user is not expected to write the generated YAML or Python files. However, `user_info.txt` must contain a complete, reviewed scientific specification. Table 1 summarizes the required information.

Table 1: Information required to define a new parameter-estimation problem.

Required information	Description
ODE system	Every integrated state must be named and have exactly one right-hand-side expression. Intermediate rates and formulas referenced by the equations must also be defined.
Dataset files and mappings	Every CSV filename and column meaning must be stated. Measured columns must map unambiguously to integrated states or derived observables, while forcing and uncertainty columns must be identified by role.
Initial conditions	A finite initial value must be provided for every integrated state. Experiment-specific overrides must identify both the dataset and the affected state.
Trainable parameters	Every fitted parameter must have a name, finite lower and upper bounds, and an explicit choice of linear or logarithmic search coordinates.
Fixed parameters	Every fixed parameter must have a name and finite numerical value. Values stated in the input specification are treated as authoritative.
Derived observables	Any simulated quantity compared with data must be defined mathematically in terms of states, parameters, and previously defined formulas.

Table 1 – continued from previous page

Required information	Description
Measured forcing inputs	Known time-dependent inputs must be associated with declared CSV columns. The current workflow supports finite forcing histories with linear interpolation.
Loss definition	The objective must be stated mathematically, including simulated-to-measured mappings, normalization, transformations, MSE or RMSE reductions, channel and experiment aggregation, weighting, penalties, and missing-data behavior.
Numerical and output requirements	Known stiffness, required integrator behavior, tolerances, and any necessary output or writeout definitions should be stated when they are scientifically important.

The loss and parameter bounds are deliberately set to be required inputs rather than choices delegated to the model.

Based on the problem provided, the model automatically chooses between an explicit or implicit high order integrator, makes an estimation on maximum solver steps that should be used, calibrates the per-state tolerances that should be used, and performs solver accuracy checks after the selection of these.

Units must already be consistent across the equations, data, initial conditions, parameter values, bounds, and objective. The framework does not infer units or perform unit conversions.

Parameter fitting uses JAX CPU devices. The requested number of processors is limited by the CPUs available to the process, and the final resource allocation is recorded with each run.

Requirements on the dataset files

The framework does not modify user data files, so each CSV must already be in a form that the loader, validator, and ODE solver can use. The requirements are summarized in Table 2.

Table 2: Requirements for CSV data supplied to the pfit workflow.

Requirement	Remarks
Comma-separated text	Files must use ordinary comma-separated fields. Tab-delimited and other separator formats are not supported by the fitting-data loader.
Required header row	New sessions require a header row. The first header must be named exactly `time`. Headerless files remain readable only for compatibility with older prepared sessions whose column meanings are supplied by an existing YAML configuration.
Time is the first column	Column 0 contains the recorded time grid. Time values must be numeric, finite, and strictly increasing. The framework does not sort or repair time values.
Minimum size	Each CSV must contain at least two numeric data rows and at least one declared non-time column.

Continued on next page

Table 2 – continued from previous page

Requirement	Remarks
Rectangular structure	Every row must contain the same number of fields, and the CSV width must match the declared column layout. A trailing comma may create an additional empty field and therefore cause a width mismatch.
Numeric data cells	Data cells must contain numeric values or permitted missing-measurement values. Repeated header rows and arbitrary non-numeric text in the body of the file are rejected rather than silently converted.
Declared column meanings	Every non-time column must have a unique, explicit role and meaning. A column may represent a measured integrated state, a measured derived observable, a measurement uncertainty, or a known forcing input. Column names and mappings must agree with the generated session configuration.
Measurement columns	A measurement column must map unambiguously to an integrated state or derived observable. The framework does not infer a mapping solely from a similar name or from the numerical values in the column.
Missing measurements	Blank cells and numeric `NaN` values may represent missing measurements. The standard generated loss masks missing measurements before normalization and residual reduction. Infinite measurement values are rejected. A required observation channel with no finite values is also rejected.
Custom loss masking	Custom and uncertainty-weighted losses are responsible for applying valid masks before logarithms, normalization, or division. The framework does not guarantee safe masking for arbitrary custom loss algebra. Nonfinite simulated trajectories are numerical failures and are never treated as missing experimental data.
Uncertainty columns	An uncertainty column must explicitly identify the measurement column to which it belongs. Its presence does not automatically select an uncertainty-weighted loss or define a statistical likelihood; the mathematical objective must state how the uncertainty is used.
Forcing columns	A forcing column represents a known input history rather than an observation to be fitted. Every forcing value must be finite, and the forcing data must cover the complete integration interval. The current workflow uses linear interpolation. Missing forcing values, extrapolation, separate forcing grids, zero-order hold, and automatic unit conversion are unsupported.
Initial conditions	Initial conditions are supplied in `user_info.txt` and recorded in the generated configuration; they are not represented by a special CSV row. Experiment-specific initial-condition overrides must be finite and keyed by integrated-state name. The user is responsible for ensuring consistency with any measured value at the initial time.

Continued on next page

Table 2 – continued from previous page

Requirement	Remarks
Multiple experiments	All CSVs in one fitting session must use the same ordered column names, roles, measurement mappings, uncertainty relationships, and units. Their row counts, time grids, time ranges, numerical values, and declared initial conditions may differ.
Consistent units	Time, measurements, forcing inputs, initial conditions, model equations, and parameter values must already use compatible units. The framework neither infers units nor performs unit conversion.
Preservation and provenance	The original CSV files are not rewritten. Each fitting run copies the selected data into its run snapshot and records the original filenames and content hashes, allowing later diagnosis to use the exact data supplied to that run.

When generated model code is executed, the time column is passed separately as the integration grid. Consequently, `dataset[:, 0]` in generated Python or JAX code refers to the first *non-time* CSV column, not to time. Users should keep this indexing convention in mind when reviewing the generated loss, forcing, and output functions.

Fitting multiple datasets simultaneously add example of file labeling for datasets

Several experimental records may be fitted simultaneously when they describe the same dynamical system under different experimental conditions. The framework estimates one shared parameter vector and uses the same equations, fixed parameters, observables, loss implementation, and numerical settings for every record. Experiments that require different fitted parameter values should instead be placed in separate sessions.

Each experiment is associated with its own CSV file and may have its own time grid, number of observations, initial and final times, initial-condition overrides, and forcing history. The problem statement must identify every file and state explicitly what differs between the experiments. For example:

```
Experiments:
- run_low.csv:  A(0) = 0.1, B(0) = 0
- run_mid.csv:  A(0) = 1.0, B(0) = 0
- run_high.csv: A(0) = 10.0, B(0) = 0

All experiments use the same equations, parameters, observables,
column meanings, units, and mathematical loss.
```

All CSV files in a session must have the same ordered column names, roles, measurement mappings, uncertainty relationships, and units. Numerical values and row counts may differ. Forcing histories may also differ between experiments, but the forcing columns and their interpolation roles must be declared consistently in every file.

The scalar loss is evaluated separately for each experiment using that record's own data, time grid, initial conditions, and forcing values. If there are N_{exp} experiments with scalar losses $\mathscr{L}_e$, the fitted objective is

$$\mathscr{L}_{\text{total}} = \frac{1}{N_{\text{exp}}} \sum_{e=1}^{N_{\text{exp}}} \mathscr{L}_e. \tag{11}$$

This is an equal-weight mean of complete experiment losses, not a pooled loss over every observation. The mathematical loss should therefore define a suitable reduction and normalization within each experiment. Otherwise, an experiment whose scalar loss is numerically much larger may dominate the optimization despite the equal experiment weights. Arbitrary framework-level experiment weights are not currently supported.

Blank values may represent individual missing measurements, subject to the masking requirements described above. However, a required observation channel with no finite measurements in an experiment is rejected by the standard generated loss. The main workflow should therefore not be assumed to support arbitrary combinations of entirely absent output channels through automatic masking.

Unless a global integration start time is explicitly configured, each experiment begins at the first timestamp in its own CSV file. If a global start time is provided, it must be valid for every record. A failed integration, nonfinite simulated trajectory, nonfinite scalar loss, or configured error-loss sentinel from any experiment invalidates that parameter candidate.

Each experiment receives its own saved trajectory:

```
result_solution_exp1.csv
result_solution_exp2.csv
...
```

The run summary records the loss for each experiment as well as the equal-experiment-mean aggregation. Run snapshots, gradient-only restarts, sloppiness analysis, and post-fit diagnosis all use the complete set of experimental records.

Requirements on the input statement

The file `inputs/user_info.txt` is the authoritative, human-readable description of the fitting problem. It should contain the scientific information listed in Table 1, while the preceding sections define the separate requirements on the CSV files.

The statement does not need to use YAML, Python, or another machine-readable format. Equations may be written in ordinary mathematical notation, and the document may combine prose, equations, lists, and short code expressions. However, it must be sufficiently explicit that the local model does not need to invent missing scientific decisions.

Names must be used consistently. States, parameters, intermediate rates, observables, and forcing inputs should have distinct names that are valid Python identifiers. Names that differ by capitalization or underscores are treated as different quantities. Every symbol used in an ODE or observable must be defined as a state, parameter, forcing input, or intermediate formula.

Numerical declarations are authoritative. Fixed parameter values, initial conditions, parameter bounds, and logscale choices written in `user_info.txt` must be preserved in the generated files. For a multi-experiment problem, the statement must name every CSV and clearly describe the initial conditions or forcing values that differ between records.

The objective must be stated mathematically. A description such as "fit the data" or "minimize the error" is insufficient. The statement must identify:

- which simulated quantity is compared with each measured column;
- whether the comparison is linear, logarithmic, or otherwise transformed;
- the normalization and weighting applied to each residual;
- whether each reduction is an MSE, RMSE, sum, or another operation;
- how channels and experiments are combined;
- how missing measurements are treated; and
- any additional penalty terms.

A useful organization is:

```
Problem:
Datasets and experiment-specific conditions:
States and initial conditions:
Trainable parameters and bounds:
Fixed parameters:
Intermediate formulas:
ODE system:
Observables and data mappings:
Forcing inputs:
Loss:
Numerical requirements:
Outputs:
```

These headings are recommendations for clarity rather than a separate input language. Sections that do not apply, such as forcing inputs or special output requirements, may be omitted.

If required information is missing or contradictory, `pfit new` reports targeted `missing_inputs` and stops. It does not conduct an interactive question-and-answer session or silently choose missing bounds, equations, initial conditions, mappings, or loss terms.

An external vendor-provided model may be used to prepare `user_info.txt` from a longer source document. The user must review the resulting equations, values, mappings, units, and objective before supplying the file to pfit. The reviewed text and CSV files are the authoritative inputs to the local workflow.

Objective function

The construction of the objective is central to the quality of the obtained estimates. The objective **must** capture all constraints on the ODE solution the user requires. Generally, for an ODE of the form $\mathbf{y} = ODE(\mathbf{x})$, the general form of a constraint is $f_i(\mathbf{x},\mathbf{y}) \leq 0$ add eq numbers etc and this is

added to the loss through a weighted linear combination of penalty functions of the form. add loss function form, add a note on relative differentiability of different kinds of losses

This requires some user expertise and insight, as an underconstrained problem implies the solution may have unanticipated or undesired dynamics, and an overconstrained problem runs the risk of an optimizer not finding a feasible solution during the global optimization stage. It should be noted that the user may choose to allow the agent to formulate the loss itself based on the problem specification. This works for simpler problems, but for more complex problems, it is advised the user specify how the loss is constructed from the ODE solution manually. Broadly, there are best practices for constructing the objective function, listed in Table.

Best Practice	**Remarks**
Every term of the overall objective should be of similar scale in magnitude (preferably close to unity)	It is recommended to divide each objective term by a scale belonging to its observable, like its own peak, value range, or measurement uncertainty. This enables more uniform estimation of parameters, and a relative importance of terms can be controlled by setting channel weights. Several assumptions in the framework are built around this scale being close to unity. link example
Weighting of samples within a trajectory	add ARC example Some trajectories to fit to might only carry relevant dynamics in small subsets of the overall time history, but by default each sample in time history may be weighted equally. This can lead to poor estimates, and the loss formulation can be weighted unequally in time to mitigate this
Scaling of value *within* a loss term	In some instances, the data being estimated can span several orders of magnitude within a single channel and trajectory. Users are advised to use logarithmic scalings to allow for for uniform contributions to the objective in such cases
Differentiability of constraints	To allow for gradient-based fine tuning, it is recommended to add penalty terms in a form that allow for differentiability. For example, rather than adding a step function add forms the user can use a sigmoid with high weight

add table label, increase spacing

Fitting multiple datasets using a single model A few additional guidelines exist when fitting a model to multiple datasets:

- The same loss function is applied to each dataset, so it has to be written to work for any of them.
- Because the datasets are averaged with equal weight, each must be normalized against its own scale rather than a constant shared by the session.
- It is recommended to average the estimate of a single dataset rather than sum, to avoid disproporionate contribution by any one term

3.1 Starting a new session

After preparing the input statement and data files, the session directory should contain the user-supplied files:

```
sessions/<session-name>/
  inputs/
    user_info.txt
    <data-file>.csv
    ...
```

The file `user_info.txt` is the reviewed scientific description of the problem, and the CSV files are the experimental records referenced by that description. The generated YAML and Python files are not starting inputs. They are created by the workflow.

Create the session with:

```
pfit new sessions/<session-name>
```

The configured local model extracts the problem in bounded stages using the input statement and CSV headers. The stages identify the selected experiments, trainable and fixed parameters, integrated states and initial conditions, equations, derived observables, measured and forcing columns, and the loss definition. If a required item is missing or contradictory, the command reports targeted `missing_inputs` and stops. It does not conduct an interactive question-and-answer session or silently invent missing equations, parameter bounds, initial conditions, data mappings, units, or loss terms.

Running example. The tutorial uses a two-state drug-response model fitted simultaneously to pulse and washout experiments. Both CSV files use the same columns,

```
time,dose_rate,signal
```

but have different forcing histories and initial conditions. The corresponding `user_info.txt` is:

```
Problem:
Fit pulse.csv and washout.csv using one shared parameter set.

Experiments and initial conditions:
pulse.csv:   D(0) = 0, R(0) = 0
washout.csv: D(0) = 0, R(0) = 0.3

CSV columns:
time: time in minutes
dose_rate: known forcing input in concentration per minute;
           use linear interpolation
signal: measured value of signal_model in arbitrary units
```

```
Trainable parameters:
k_elim:   bounds [0.01, 10], logarithmic
k_on:     bounds [0.001, 10], logarithmic
k_off:    bounds [0.001, 10], logarithmic
baseline: bounds [50, 150], linear
gain:     bounds [1, 200], linear

Fixed parameters:
None.

ODE system:
dD/dt = dose_rate - k_elim*D
dR/dt = k_on*D*(1 - R) - k_off*R

Observable:
signal_model = baseline + gain*R
CSV column signal measures signal_model.

Loss:
For each experiment e, use only finite signal measurements.

scale_e = max(abs(finite signal values in experiment e))

L_e = sqrt(mean(
    ((signal_model_e - signal_e)/scale_e)^2
))

The total loss is the equal-weight mean:
L = (L_pulse + L_washout)/2.

Outputs:
Write time, D, R, measured signal, and signal_model.
```

This statement supplies the scientific decisions required by the workflow. The local model is not expected to choose missing equations, parameter bounds, initial conditions, data mappings, units, or loss terms. The measured CSV column `signal` is valid here because the statement explicitly maps it to the derived observable `signal_model`. In general, measured columns should map unambiguously to declared integrated states or derived observables, and forcing or uncertainty columns should be identified by role.

Generated files. If extraction and validation succeed, `pfit new` creates:

```
sessions/<session-name>/
  inputs/
    user_info.txt
    user_input.yaml
    pulse.csv
```

```
    washout.csv
  generated/
    user_model.py
    agent_logs/
```

The file `inputs/user_input.yaml` contains the structured experiment definitions, column roles, initial conditions, parameter bounds, state ordering, numerical settings, and output configuration. Generated configurations also include provisional numerical automation settings used later by `pfit jax`, including automatic integrator selection, automatic `max_steps` estimation, bounded solver-step recovery, tolerance calibration, and solver-accuracy checks. These settings are subject to later validation and may be adjusted during JAX generation.

The file `generated/user_model.py` contains the readable NumPy implementation of the ODEs, forcing interpolation, observables, loss, and output calculation. Detailed model calls and workflow events are stored under `generated/agent_logs/`.

Before these files are accepted, the framework checks the CSV headers, declared names, numeric values, parameter bounds, initial conditions, equation symbols, column mappings, generated Python interfaces, and selected source-consistency conditions. These are checked against the original problem statement to ensure consistency and to avoid information loss during the translation. Bounded repair attempts may correct eligible expression or response-format errors.

Passing `pfit new` means that the generated artifacts satisfy the checks implemented for new-session extraction. Before continuing, the user should compare the generated files with the original specification and verify the selected datasets, parameter bounds, state ordering, initial conditions, equations, forcing interpolation, observable mappings, loss, output definitions, and numerical settings.

After intentionally changing the source statement, regenerate the session with:

```
pfit new sessions/<session-name> --overwrite
```

This replaces the generated new-session artifacts while preserving the user-supplied `user_info.txt` and CSV files.

3.2 Checking the session

After reviewing the files produced by `pfit new`, check the session with:

```
pfit check sessions/<session-name>
```

This command reads the generated YAML, source model, and every declared CSV file. The framework first performs deterministic checks by parsing these files and comparing their declarations. It verifies that the data dimensions and headers match the configured columns, parameter values and bounds are valid, initial conditions are complete, model names are consistent, and the source model uses the expected interfaces and column indices. Because the time column is passed separately to generated model code, these checks also help catch incorrect uses of dataset indexing: `dataset[:, 0]` refers to the first non-time CSV column, not to time.

The stated loss is also compared with the generated implementation. For example, if `user_info.txt` requests an RMSE, logarithmic residuals, uncertainty weighting, finite-value masking, a particular

normalization, or a specific experiment aggregation, the generated loss must contain the corresponding operation. However, if an explicit user-supplied loss is provided, it overrules any automated loss function generation. The check surfaces potential scaling issues as warnings, which are crucial to fit quality.

The deterministic checks also look for several common failure modes: non-smooth or branch-like dynamics that may be difficult to differentiate, measured columns with large dynamic range, and loss implementations that appear to violate any explicit loss instructions in `user_info.txt`.

For the running example, the framework verifies that both CSV files use the same ordered columns; that `dose_rate` is declared as a finite forcing input; that `signal` maps to the derived observable `signal_model`; and that the two experiments retain their specified initial conditions. It also checks that the generated loss uses finite signal measurements, applies the stated per-experiment normalization, calculates an RMSE for each experiment, and preserves the equal-experiment aggregation.

The report is written to:

```
sessions/<session-name>/generated/user_input_check.txt
```

After performing a check, three categories of errors are surfaced. Critical errors must be corrected before continuing. Warnings identify limitations or potentially fragile assumptions, while recommendations identify choices that the user may wish to reconsider.

By default, the configured local model may also review the session description, generated source, and deterministic evidence for semantic inconsistencies. Because model-generated criticism can have variance in it's finding, any issues reported by the semantic analysis are retained as warnings rather than allowed to veto an otherwise valid session.

To omit this semantic review and run only deterministic checks, use:

```
pfit check sessions/<session-name> --deterministic-only
```

After JAX translation, the command

```
pfit check sessions/<session-name> --ready
```

additionally verifies that the generated JAX script has the required interface and was produced from the current YAML and source model. Source hashes recorded during translation are used to detect later edits. The readiness check does not regenerate the JAX translation or repeat the full numerical validation performed by `pfit jax`. The same readiness checks are repeated automatically before fitting.

It should be noted that passing the checks confirms that the session satisfies the implemented structural, numerical, and consistency rules for that stage. It does not establish scientific correctness or model optimality for the data, parameter sloppiness, optimizer convergence, or global optimality.

3.3 Generating the JAX model

Once the session has passed the checks described above, generate the executable JAX model by running:

`pfit jax sessions/<session-name>`

This command converts the readable NumPy model in `generated/user_model.py` into the differentiable JAX implementation used for parameter estimation. The workflow breaks the code generation into smaller steps and completes translation. The resulting script is written to:

`sessions/<session-name>/generated/generated_script.py`

A simplified excerpt of the transformed JAX system for this example is:

```
@jax.jit
def user_defined_system(t, y, other_args):
    constants = other_args["constants"]
    trainable_variables = other_args["trainable_variables"]
    dataset = constants["dataset"]
    t_eval = constants["t_eval"]

    k_elim = unscale_value(trainable_variables[0], 0.01, 10.0, True)
    k_on = unscale_value(trainable_variables[1], 0.001, 10.0, True)
    k_off = unscale_value(trainable_variables[2], 0.001, 10.0, True)
    baseline = unscale_value(trainable_variables[3], 50.0, 150.0, False)
    gain = unscale_value(trainable_variables[4], 1.0, 200.0, False)

    D = y[0]
    R = y[1]
    dose_rate = jnp.interp(t, t_eval, dataset[:, 0])

    dDdt = dose_rate - k_elim * D
    dRdt = k_on * D * (1.0 - R) - k_off * R
    return jnp.array([dDdt, dRdt])

@jax.jit
def _compute_loss_value(constants, trainable_variables,
                        solution_time, solution):
    dataset = constants["dataset"]
    baseline = unscale_value(trainable_variables[3], 50.0, 150.0, False)
    gain = unscale_value(trainable_variables[4], 1.0, 200.0, False)

    R = solution[:, 1]
    signal_model = baseline + gain * R
    signal = dataset[:, 1]
    mask = jnp.isfinite(signal)
    scale = jnp.max(jnp.abs(signal[mask]))
```

```
    residual = (signal_model[mask] - signal[mask]) / scale
    return jnp.sqrt(jnp.mean(residual**2))
```

This excerpt omits the surrounding integration scaffold, error handling, writeout routine, and multi-experiment aggregation, but it shows the main transformation: parameters are mapped from optimizer coordinates to physical values, the forcing input is interpolated from the non-time dataset columns, the right-hand side is written with `jax.numpy`, and the stated normalized RMSE is evaluated on finite measurements.

After generation, the framework first checks that the script provides the required runtime interface for integration, loss evaluation, and output writing. It then imports the script, runs the model for each dataset, verifies that trajectories and losses are finite, and evaluates the generated code on some sampled parameter points. These sampled points are chosen in the normalized search coordinates used by the optimizers, including logarithmic coordinates for parameters marked as log-scaled.

The JAX implementation is compared with the readable NumPy source model at sampled states and parameter values. The comparison checks differential-equation values, observable predictions, output columns, and losses where applicable. These tests are intended to detect accidental changes introduced during translation, such as altered parameter names, incorrect column indices, missing observable transformations, or losses that no longer match the source model.

Numerical readiness checks. The JAX stage also performs numerical checks needed before fitting. For fresh generated configurations, the framework may select an integrator from the data, estimate an initial `max_steps` budget, test solver coverage across sampled parameter points, calibrate state-scaled tolerances, and compare predictions and losses against integrations run with tighter tolerances. These checks are deterministic and are recorded in JSON reports under `generated/`.

The framework also adaptively tunes some heuristics of the ODE solver at this stage. For instance, if too few sampled parameter points integrate successfully because the solver reaches its step limit, the workflow may increase `max_steps` up to the configured recovery ceiling and rerun the same validation. If recovery succeeds, the accepted step budget is written back to the YAML configuration. Similarly, proposed state-scaled tolerances or population-stage relaxations are accepted only when they preserve finite losses on a meaningful fraction of sampled candidates. Solver-accuracy checks then compare selected feasible probes with tighter reference solves, until the tolerance gets too tight for successful integration.

Repair and source freshness. If a translation comparison or interface check fails, the local model does a targeted repair on the fragment responsible for the failure. The number of repair attempts is limited. A session is not marked ready if the generated model still fails its checks.

Before fitting, the user should inspect `generated_script.py` and the generated reports and confirm that the equations, parameter names, observable definitions, loss function, output columns etc. match the intended workflow

Finally, verify that the complete session is ready to run:

```
pfit check sessions/<session-name> --ready
```

3.4 Running the fit

After the JAX model has been generated and the session passes readiness checks, run parameter estimation with:

```
pfit run sessions/<session-name>
```

This stage does not require the local LLM model, and can be run repeatedly on a CPU-only device. It uses the accepted YAML configuration, CSV files, and `generated_script.py`. Before fitting starts, the same deterministic readiness checks are repeated to ensure that the generated script is current.

Each invocation creates a new preserved run directory under:

```
sessions/<session-name>/outputs/
```

The run directory contains a snapshot of the input YAML, selected data files, generated JAX script, runtime provenance, fitted parameters, optimization history, fitted trajectories, and summary files. These snapshots allow a completed result to be diagnosed later even if the working session files are changed.

The default run performs population-based global exploration followed by gradient-based refinement. The population stage searches the bounded parameter space in the normalized coordinates defined by the YAML configuration. The gradient stage starts from the best population result and refines it using the differentiable JAX loss.

For an additional refinement starting from a previous run, use:

```
pfit run sessions/<session-name> gradient-only --from-run <run-id>
```

By default, the run also performs post-fit sloppiness analysis at the final parameter estimate. A completed run indicates that the configured optimization procedure finished and wrote its artifacts.

3.5 Diagnosing a completed fit

After parameter estimation has finished, examine the result with:

```
pfit diagnose sessions/<session-name> <run-id>
```

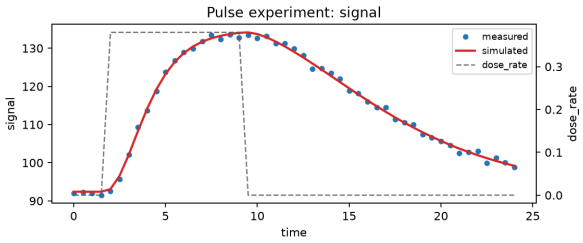
The run identifier is the name of a directory under `sessions/<session-name>/outputs/`. If it is omitted, the most recent completed run is selected.

For each dataset, the command recreates the fitted trajectory and compares it with the measurements. In the running example, separate plots are produced for the pulse and washout experiments, as seen in Figure X. These show the measured signal together with the fitted `signal_model`.

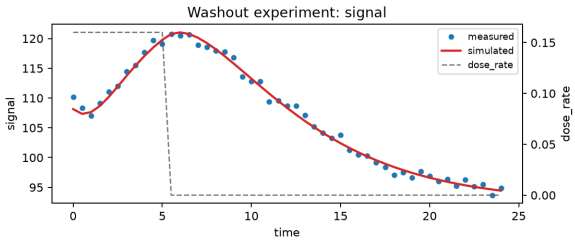
The diagnosis reports residual summaries, including root-mean-square error, mean absolute error, bias, largest absolute residual, and time-pattern evidence. The report also checks whether fitted parameters are close to their bounds, whether optimizer logs suggest that improvement was still occurring, and whether the run used a full fit or a gradient-only restart. If post-fit sloppiness analysis was performed during `pfit run`, the diagnosis includes its status and summary, and a plot of the sloppiness diagnosis as visible in Figure X.

For the running pulse/washout example, representative diagnostic plots are shown in Figure 1.

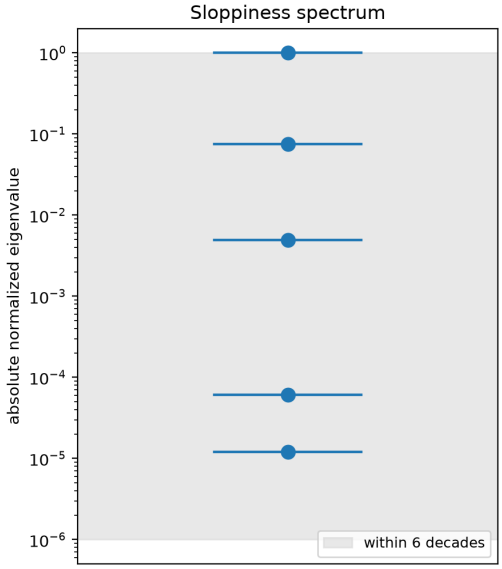
A representative excerpt from the file `fit_diagnosis.txt` for this example is:



(a) Pulse experiment.



(b) Washout experiment.



(c) Post-fit sloppiness spectrum.

Figure 1: Example diagnostic outputs for the pulse/washout running example.

```
Fit diagnosis

Artifacts:
- final_design_point.csv: present
- result_solution_exp1.csv: present
- result_solution_exp2.csv: present
- de_fitting.log: present
- NODE_fitting.log: present
- fitting_error.txt: missing

Optimizer summary:
- DE: 120 iterations, first loss 8.0936E-02,
      best loss 5.7755E-03, final loss 5.7755E-03
- NODE: 21 iterations, first loss 5.7754E-03,
        best loss 5.7753E-03, final loss 5.7753E-03
```

```
Experiments: 2
Aggregation: equal_experiment_mean
- Experiment 1: pulse.csv; loss=0.005219267638936577;
  output=result_solution_exp1.csv
- Experiment 2: washout.csv; loss=0.006331386422963499;
  output=result_solution_exp2.csv

Scientific diagnosis:
- Status: ok
- Refinement stopped at its iteration budget.
- [BUDGET] Refinement stopped at its iteration budget.
  Next: Assess the final gradient and trajectory plots before
  deciding whether a gradient-only restart is worthwhile.
- Figure: fit_exp1.png
- Figure: fit_exp2.png
- Detailed evidence: scientific_diagnosis.json
```

When sloppiness analysis is enabled, a representative `sloppiness_report.txt` excerpt is:

```
Post-fit sloppiness
Status: ok
Method: finite-difference
Coordinates: log10(k_elim), log10(k_on), log10(k_off), baseline, gain
Verdict: no weak curvature directions at the specified relative floor
Weak modes: 0
Negative modes: 0
Normalized gradient infinity norm: 5.63383e-06
Eigenvalues (largest to smallest):
  mode 1: 1.68783850e+01
  mode 2: 1.26587665e+00
  mode 3: 8.25273064e-02
  mode 4: 1.03427087e-03
  mode 5: 2.01732936e-04
Positive spectrum span: 4.92255 decades
STIFFEST direction: k_off(-0.64), k_elim(-0.63),
                    k_on(+0.43), baseline(+0.02)
SLOPPIEST direction: gain(+1.00), baseline(-0.08),
                     k_on(-0.02), k_elim(+0.01)
Warning: Local curvature only: not proof of structural/global
identifiability or convergence.
```

By default, the framework may ask the configured model to summarize the computed evidence and suggest next steps. This returns a semantic interpretation of the results. To produce only the deterministic report without using the local model, which may be desired if GPU access is only transient, use:

```
pfit diagnose sessions/<session-name> <run-id> --deterministic-only
```

Table 3: Memory requirements, average completion time (all 5 stages, passed cases only) and completion rates for attempted ODE estimation problems

Model	Memory Req. (GB)	Mean completion time (s)	Pass	Fail	Success Rate
Qwen2.5-Coder 32B (A)	19.9	110.0	15	8	71.4%
Qwen2.5-Coder 14B (B)	9.0	93.8	13	10	61.9%
Qwen3.8-27B (C)	17.7	235.6	19	4	90.5%

4 Model Performance

add links to full fits on repo

This section discusses the tradeoffs of using different LLMs for the process.The workflow is evaluated on a suite of ODE parameter-estimation tasks using three locally served Ollama models. The focus is on problems with experimental data, and several examples from biological sciences are taken directly from an external, community-curated benchmark collection (the PEtab benchmark suite [17]).

The framework is evaluated using three different LLM models, with the comparison run on an NVIDIA H100 80GB HBM3 GPU. The Ollama request configuration used a 32k token context window, temperature 0.1, and a 12k token generation budget. The temperature value, which controls randomness, is kept deliberately low for this application which requires high reproducibility. Table 3 discusses memory approximate memory consumed for each model, along with their success rates on tested problems. The tests show that using newer and larger models has a higher success rate, but has larger hardware requirements. For example, Qwen2.5-Coder 14B can also be run for these problems on an Apple M4.

Table 4 summarizes the observed completion behavior, where identical tasks were provided to 3 different models. Qwen3.8-27B completed 19 of 23 tasks (90.5%), compared with 15 of 23 for the 32B baseline (71.4%) and 13 of 23 for the 14B baseline (61.9%). The Qwen3.8 model was slower in wall-clock time, partly because it is a thinking model and its reasoning tokens are included in the generation budget, but it was substantially more reliable on the larger biochemical and multi-experiment cases. The Qwen2.5-Coder 14B model was fast and often adequate for small canonical systems, but it failed on several larger multi-experiment or biochemical prompts during `pfit new`.

The retained examples span a deliberately broad range of ODE fitting tasks. They include low-dimensional textbook dynamics, stiff chemical kinetics, oscillatory systems, biochemical signaling networks, pharmacokinetics, hysteresis, measured forcing functions, derived observables, and multi-experiment shared-parameter fits. The diversity of examples demonstrates that the workflow works well in qualitatively different modeling regimes.

Table 4: End-to-end local workflow completion by models across a diversity of ODE tasks. A check mark denotes successful completion of all five pfit stages, but does not include RMSE as a benchmark, since that depends on suitability of model dynamics and noise levels of data. Models A,B,C are as described in Table 3

Case	Model A	Model B	Model C	Source or model family
ARC fitting	✓	✓	✓	Accelerating-rate calorimetry battery example [18]
Boehm STAT5	✓	✓	✓	STAT5 signaling model [19]
Lotka–Volterra	✓	✓	✓	Predator–prey equations [20, 21]
MAPK cascade	✓	✓	✓	Kholodenko MAPK cascade [22]
Oregonator	✓	✓	✓	Field–Noyes/Tyson BZ reaction model [23, 24]
Piezo Bouc–Wen	✓	✗	✓	Bouc–Wen hysteresis family [25, 26]
Robertson	✓	✓	✓	Stiff chemical kinetics model [27]
Sliding basepoint	✗	✓	✓	Frictional sliding-basepoint example [28]
Theophylline	✓	✓	✓	One-compartment pharmacokinetic example [29]
Van der Pol	✓	✓	✓	Nonlinear oscillator [30]
Cascaded tanks	✓	✗	✓	Two-tank process-control example [31]
Multi-exponential decay	✓	✓	✓	Synthetic multi-experiment decay example [32]
Sneyd IP_3 receptor	✗	✓	✓	IP_3 receptor model [33]
Beer indigoidine	✓	✓	✓	Indigoidine production dataset [34]
Raia IL-13	✓	✗	✓	IL-13/JAK2/STAT5 lymphoma model [35]

A few representative completed fits are shown in Figure 2. The Oregonator example tests a stiff oscillatory chemical system with only a subset of states observed; the sliding-basepoint case tests a nonsmooth frictional model with derived force and displacement observables; the cascaded-tanks example tests a measured forcing input.

The completed examples are included in the repository under `sessions/` at the repo. Each curated session contains the user prompt and data under `inputs/`, the generated model artifacts under `generated/`, and at least one completed fitting run under `outputs/`. The `outputs/` directory for each retained case also contains one or more `fit_exp*.png` figures showing the fitted trajectory against the data. These sessions are intended to serve both as examples for users and as reproducibility evidence for the local-agent workflow.

It should be noted that successful execution of the workflow is not the same thing as proving that the proposed ODE is a perfect description of the system. In several cases the framework produces a stable, finite, and visually useful fit while the residual structure still shows systematic mismatch. This can occur due to external factors such as the model being an imperfect representation of system dynamics, measurement noise and more.

5 Summary and Conclusions

A note on model appropriateness

The framework finds a good solution based on equations, data, losses and constraints describe the problem. One of the main purposs of the framework is to allow users to easily fit to noisy, real world data. In most applications, this means that the model is an approximate representation of real world dynamics, and the framework cannot guarantee perfect fits.

Competing Interests

The author has no conflicts of interest to declare that are relevant to the content of this chapter.

References

References

[1] Saakaar Bhatnagar. An agentic ai workflow to simplify parameter estimation of complex differential equation systems, 2025. URL `https://arxiv.org/abs/2509.07283`.

[2] Andreas Raue, Clemens Kreutz, Thomas Maiwald, Julie Bachmann, Marcel Schilling, Ursula Klingmuller, and Jens Timmer. Structural and practical identifiability analysis of partially observed dynamical models by exploiting the profile likelihood. *Bioinformatics*, 25(15): 1923–1929, 2009. doi: 10.1093/bioinformatics/btp358.

[3] Ryan N. Gutenkunst, Joshua J. Waterfall, Fergal P. Casey, Kevin S. Brown, Christopher R. Myers, and James P. Sethna. Universally sloppy parameter sensitivities in systems biology models. *PLoS Computational Biology*, 3(10):e189, 2007. doi: 10.1371/journal.pcbi.0030189.

[4] Mohammad S Khan, Sankalp Sinha, Talha U Sheikh, Didier Stricker, Sk A Ali, and Muhammad Z Afzal. Text2cad: Generating sequential cad designs from beginner-to-expert level text prompts. *Advances in Neural Information Processing Systems*, 37:7552–7579, 2024.

[5] Yandong Guan, Xilin Wang, Ximing Xing, Jing Zhang, Dong Xu, and Qian Yu. Cad-coder: Text-to-cad generation with chain-of-thought and geometric reward. *Advances in Neural Information Processing Systems*, 38:59765–59789, 2026.

[6] Yuxuan Chen, Xu Zhu, Hua Zhou, and Zhuyin Ren. Metaopenfoam: an llm-based multi-agent framework for cfd. *arXiv preprint arXiv:2407.21320*, 2024.

[7] Ling Yue, Peijing Xu, Tingwen Zhang, Nithin Somasekharan, and Shaowu Pan. Foamgpt: Fine-tuning large language model for agentic automation of cfd simulations with openfoam. *Advances in Neural Information Processing Systems*, 2025.

[8] Zhaoyue Xu, Long Wang, Chunyu Wang, Yixin Chen, Qingyong Luo, Hua-Dong Yao, Shizhao Wang, and Guowei He. Cfdagent: A language-guided, zero-shot multi-agent system for complex flow simulation. *Physics of Fluids*, 37(11), 2025.

[9] Yuxuan Chen, Xu Zhu, Hua Zhou, and Zhuyin Ren. Metaopenfoam 2.0: Large language model driven chain of thought for automating cfd simulation and post-processing. *arXiv preprint arXiv:2502.00498*, 2025.

[10] Donggeun Park, Hyeonbin Moon, and Seunghwa Ryu. A self-correcting multi-agent llm framework for language-based physics simulation and explanation. *npj Artificial Intelligence*, 2, 2026. doi: 10.1038/s44387-025-00057-z.

[11] Guillaume Damblin, Mathieu Couplet, and Bertrand Iooss. Numerical studies of space filling designs: Optimization of latin hypercube samples and subprojection properties. *Journal of Simulation*, 7(4):276–289, 2013.

[12] James Kennedy and Russell Eberhart. Particle swarm optimization. In *Proceedings of ICNN'95 - International Conference on Neural Networks*, volume 4, pages 1942–1948, 1995. doi: 10.1109/ICNN.1995.488968.

[13] Rainer Storn and Kenneth Price. Differential evolution – a simple and efficient heuristic for global optimization over continuous spaces. *Journal of Global Optimization*, 11(4):341–359, 1997.

[14] Atilim Gunes Baydin, Barak A Pearlmutter, Alexey Andreyevich Radul, and Jeffrey Mark Siskind. Automatic differentiation in machine learning: a survey. *Journal of machine learning research*, 18(153):1–43, 2018.

[15] Yingbo Ma, Vaibhav Dixit, Michael J Innes, Xingjian Guo, and Chris Rackauckas. A comparison of automatic differentiation and continuous sensitivity analysis for derivatives of differential equation solutions. In *2021 IEEE high performance extreme computing conference (HPEC)*, pages 1–9. IEEE, 2021.

[16] Ryan N Gutenkunst, Joshua J Waterfall, Fergal P Casey, Kevin S Brown, Christopher R Myers, and James P Sethna. Universally sloppy parameter sensitivities in systems biology models. *PLoS computational biology*, 3(10):e189, 2007.

[17] Helge Hass, Carolin Loos, Elba Raimúndez-Álvarez, Jens Timmer, Jan Hasenauer, and Clemens Kreutz. Benchmark problems for dynamic modeling of intracellular processes. *Bioinformatics*, 35(17):3073–3082, 2019.

[18] M. N. Richard and J. R. Dahn. Predicting electrical and thermal abuse behaviours of practical lithium-ion cells from accelerating rate calorimeter studies on small samples in electrolyte. *Journal of Power Sources*, 79:135–142, 1999.

[19] M. E. Boehm et al. Stat5a/stat5b signaling model. *Reference metadata to verify*, 2014. Placeholder entry: verify exact title, author list, journal, and pages before submission.

[20] Alfred J. Lotka. *Elements of Physical Biology*. Williams and Wilkins, 1925.

[21] Vito Volterra. Fluctuations in the abundance of a species considered mathematically. *Nature*, 118:558–560, 1926.

[22] Boris N. Kholodenko. Negative feedback and ultrasensitivity can bring about oscillations in the mitogen-activated protein kinase cascades. *European Journal of Biochemistry*, 267: 1583–1588, 2000.

[23] Richard J. Field and Richard M. Noyes. Oscillations in chemical systems. iv. limit cycle behavior in a model of a real chemical reaction. *The Journal of Chemical Physics*, 60:1877–1884, 1974.

[24] John J. Tyson. A quantitative account of oscillations, bistability, and traveling waves in the belousov-zhabotinskii reaction. *Annals of the New York Academy of Sciences*, 452:109–122, 1985.

[25] Robert Bouc. Modèle mathématique d'hystérésis. *Acustica*, 24:16–25, 1971.

[26] Y. K. Wen. Method for random vibration of hysteretic systems. *Journal of the Engineering Mechanics Division*, 102:249–263, 1976.

[27] H. H. Robertson. The solution of a set of reaction rate equations. *Numerical Analysis: An Introduction*, pages 178–182, 1966.

[28] Jean Jacques Moreau. Unilateral contact and dry friction in finite freedom dynamics. In *Nonsmooth Mechanics and Applications*, pages 1–82. Springer, 1988.

[29] José C. Pinheiro and Douglas M. Bates. *Mixed-Effects Models in S and S-PLUS*. Springer, 2000.

[30] Balthasar van der Pol. On relaxation-oscillations. *The London, Edinburgh, and Dublin Philosophical Magazine and Journal of Science*, 2(11):978–992, 1926.

[31] Maarten Schoukens, Per Mattsson, Torbjörn Wigren, and Jean-Philippe Noël. Cascaded tanks benchmark combining soft and hard nonlinearities. In *Workshop on Nonlinear System Identification Benchmarks*, Brussels, Belgium, 2016.

[32] Gaspard Riche de Prony. Essai expérimental et analytique sur les lois de la dilatabilité de fluides élastiques et sur celles de la force expansive de la vapeur de l'eau et de la vapeur de l'alkool, à différentes températures. *Journal de l'École Polytechnique*, 1:24–76, 1795.

[33] James Sneyd and Jean-François Dufour. A dynamic model of the type-2 inositol trisphosphate receptor. *Proceedings of the National Academy of Sciences*, 99:2398–2403, 2002.

[34] M. Beer et al. Indigoidine production model. *Molecular BioSystems*, 2014. Placeholder entry: verify exact title, author list, and pages before submission.

[35] V. Raia et al. Dynamic mathematical modeling of il-13-induced signaling in hodgkin and primary mediastinal b-cell lymphoma. *Cancer Research*, 2011. Placeholder entry: verify exact title, author list, volume, and pages before submission.

Tables

Table 5: Suggested input files for the agentic parameter estimation workflow.

File	Purpose
`problem.xml`	Structured definition of states, parameters, data, and settings
`model.py`	User-editable model and observation functions
`data.csv`	Observation data and time points
`config.yaml`	Solver, optimizer, output, and reproducibility settings

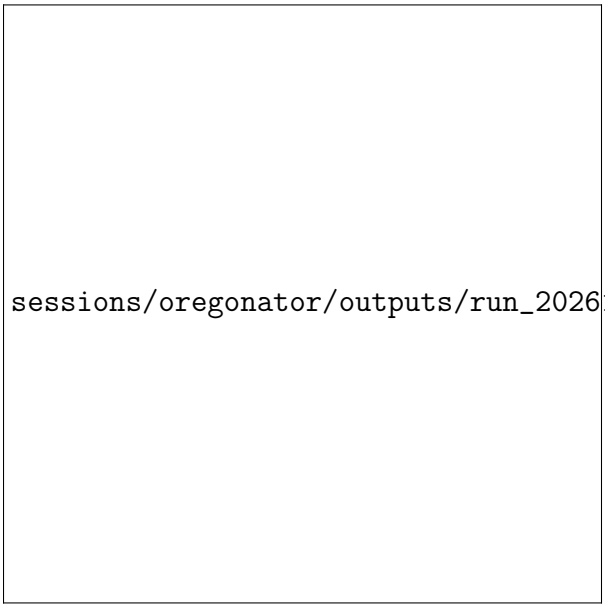
Figure Legends

Fig. 1. End-to-end agentic parameter estimation workflow. The user supplies a structured problem definition and a Python model skeleton. The agentic workflow validates the inputs, generates a differentiable calibration pipeline, runs global exploration and gradient-based refinement, and exports fitted parameters, diagnostics, and reproducible code.

Fig. 2. Example structure of the problem specification file. The schema records state variables, parameters, bounds, initial conditions, observation mappings, data paths, solver settings, and optimizer settings.

Fig. 3. Two-stage optimization strategy. A global search explores the bounded parameter space and identifies candidate regions, after which gradient-based refinement improves the best candidates using differentiable model evaluations.

Fig. 4. Example diagnostic outputs from a fitted ODE model. Suggested panels include model trajectories against observed data, residuals over time, fitted parameter values relative to bounds, and convergence history.



(a) Oregonator oscillatory chemistry.



(b) Sliding-basepoint friction model.



(c) Cascaded tanks with measured forcing.



(d) Sneyd IP_3 receptor, one of nine experiments.

Figure 2: Representative measured-versus-fitted outputs from the retained session examples. The plotted files are produced by the same `pfit diagnose` workflow used for all completed sessions.